

Experiment 03 – Advanced SQL Queries

Aim

To understand and implement advanced SQL queries using correlated subqueries, Common Table Ex-pressions (CTEs), window functions, ranking functions, aggregate functions, and update statements.

Question 1

You are given with the Employee schema; the task is to find the highest score for each respective department. For eg: department D1 have the highest score as 5.28. Hence in the output, final updated score for D1 should be 5.28. Same with the other departments i.e., D2, D3 and D4.

Answer

```
project_1=# Update Employee e
Set Scores = (
select max(Scores)
from Employee
where
Dept= e.Dept);
UPDATE 8
```

Output

```
project_1=# Select * from Employee;
eid | dept | scores
-----+-----+-----
 1 | D1   |  5.28
 2 | D1   |  5.28
 3 | D1   |  5.28
 4 | D2   |  8.00
 5 | D1   |  5.28
 6 | D2   |  8.00
 7 | D3   |  9.00
 8 | D4   | 10.20
(8 rows)
```

Question 2

A company maintains records of products sold by different salespersons. The management wants to prepare a sales performance report by identifying the salesperson(s) who generated the highest overall sales revenue. If multiple salespersons have the same highest total sales amount, all of them must be included in the final report. Return the seller ID(s) satisfying this condition.

Answer

```
# SELECT seller_id
# FROM Orders
# GROUP BY seller_id
# HAVING SUM(amount) = (
#     SELECT MAX(max_amt)
#     FROM (
#         SELECT seller_id,
#                SUM(amount) AS max_amt
#         FROM Orders
#         GROUP BY seller_id
#     ) AS seller_totals
# );
```

```
seller_id
-----
          3
(1 row)
```

Question 3

A company maintains employee salary records for different departments. The management wants to prepare a report containing employees whose salaries fall among the top three highest distinct salary values within their respective departments. If multiple employees receive the same qualifying salary, all such employees must be included in the final report. Display the department name, employee name, and salary for every qualifying employee

Answer

```
SELECT
    d.Name AS Department,
    e.Name AS Employee,
    e.Salary
FROM Employee e
JOIN Department d
    ON e.DepartmentId = d.ID
WHERE (
    SELECT COUNT(DISTINCT Salary)
    FROM Employee
    WHERE DepartmentId = e.DepartmentId
    AND Salary > e.Salary
) < 3
ORDER BY d.Name, e.Salary DESC;
```

Output

```
department | employee | salary
-----+-----+-----
IT          | Max      | 90000
IT          | Joe      | 85000
IT          | Randy    | 85000
IT          | Will     | 70000
Sales       | Henry    | 80000
Sales       | Sam      | 60000
(6 rows)

project_1=#
```

Learning Outcome

After performing this experiment, I was able to:

- Perform update operations using correlated subqueries.
- Retrieve top-ranked records within groups using nested SQL queries.
- Apply aggregate functions such as MAX() and SUM().
- Use GROUP BY and HAVING clauses effectively.
- Solve advanced SQL problems involving departmental analysis and sales reporting.
- Improve understanding of correlated subqueries and ranking-based SQL queries